基于 PYNQ-Z2 的实时音频 BPM 检测系统

摘要

本作品基于 Xilinx PYNQ-Z2（Zynq-7020, xc7z020clg400-1）平台，设计并实现了一套实时音频节拍速率（BPM, Beats Per Minute）检测系统。总体架构可以概括为“硬件负责搬运、软件负责计算”：板载 ADAU1761 音频编解码器（codec）经 I2S 将麦克风拾取的音频数字化，再由 PL 侧 AXI 数据通路与 AXI-DMA 连续搬运到 DDR，PS 端则基于 PYNQ 运行 Python/NumPy 来完成 BPM 检测算法与实时双波形可视化。

硬件链路由 ADAU1761 codec、axis_data_fifo 缓冲、 segment_stream 字数闸门、HLS 综合的自定义 bpm_detect 检测 IP 和 AXI-DMA 组成。软件层完成了 I2C 总线自动探测与 codec 寄存器配置、DMA 缓冲管理、24bit 补码解包，以及基于 Jupyter 的实时网页双波形显示。

开发中先后克服了“DMA 收帧卡死”“codec 无信号/偶发 I2C 超时”“阈值削拍漏拍”“静音期误报假 BPM”等工程问题。系统在约 46.3kHz 采样率下，对包含底鼓、镲片的音乐节奏识别表现良好，静音时 BPM 自动归零；检测算法基于 Pan-Tompkins 能量检测与区间众数统计，结构上可迁移至心率、搏动检测等场景。

第一部分 作品概述

1.1 功能与特性

① 实时音频 BPM 检测：对麦克风拾取的音频流逐样本实时处理，连续输出当前节拍速率。

② 原始音频波形 + BPM 曲线实时双显示：基于 Jupyter/网页前端，同步刷新音频时域波形与随时间变化的 BPM 曲线。

③ 能量包络与节拍计数监控：通过 IP 寄存器实时读取包络能量与累计节拍数，便于调试与标定。

④ 静音识别与自动归零：无有效节拍超过约 2s 时，输出 BPM 归零，避免“没在响却报数”的情况出现。

⑤ 自适应阈值：基于短窗能量包络与慢均值基线自动跟踪，对音量变化具有一定鲁棒性（系统在干扰、异常时仍能稳定正常工作，即鲁棒性强）。

⑥ 音乐节奏检测：对包含底鼓、镲片的真实音乐/打击乐节奏识别表现良好，抗 off-beat 误检。

1.2 应用领域

本系统本质是一个“实时音频节奏感知”原型，可作为一个通用的节拍检测平台应用于不同场景：

① 音乐创作与播放：卡拉 OK、混音、DJ 打碟的节拍对齐，也可为乐器练习提供节拍提示；

② 运动与健身：根据音乐节拍生成跑步、器械等运动节奏提示；

③ 医疗健康：所用的 Pan-Tompkins 能量检测算法与 ECG 中 QRS 波检测同构，可较低成本地迁移至心率、搏动检测；

④ 教育与科研：可作为 FPGA 实时数字信号处理、HLS 设计与软硬件协同（PS/PL）的教学与原型验证平台；

⑤ 嵌入式音频分析：为低功耗穿戴、车载娱乐等实时音频事件检测场景提供软硬件参考实现。

1.3 主要技术特点

① 软硬件协同架构：PL 专职“搬运”（FIFO/segment 闸门/AXI-DMA），PS 专职“计算与可视化”，二者解耦，兼顾实时性与算法灵活性。

② Pan-Tompkins 能量检测链路：差分（高通/去趋势）→ 平方（强化瞬态）→ 短窗移动平均包络 → 自适应阈值 → 不应期 → 峰-峰间隔，与 ECG QRS 检测同构。

③ emode 区间众数结算：以峰-峰间隔直方图 + 指数遗忘取众数替代“历史平均”，抑制音乐 off-beat 打击乐引起的 BPM 翻倍误检。

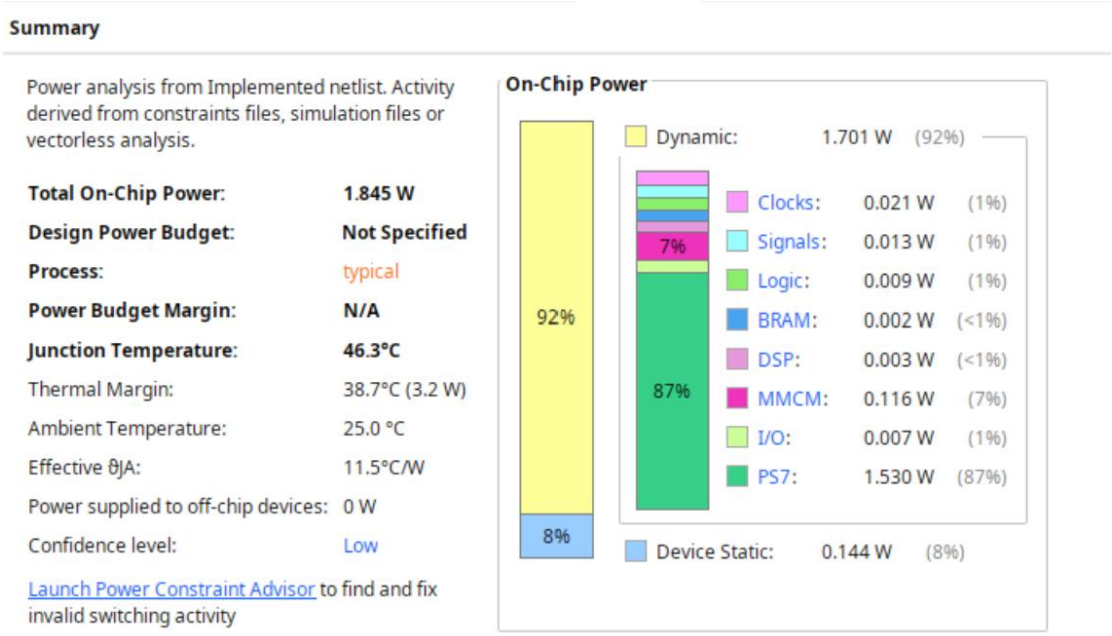
④ 自定义 HLS IP（bpm_detect）：单进程无限流式（ap_ctrl_none），输出寄存器化、关键路径时序优化，II=4 实时吞吐。

1.4 主要性能指标

项目	指标/实现	说明
采样率	≈46.3 kHz	由板上 150/120BPM 实测间隔反推校准
ADC 分辨率	24 bit（单端 MIC）	ADAU1761 内置 ADC，I2S

		从模式
BPM 检测范围	60 ~ 180	由不应期与区间上限约束
检测方式	PL freerun 实时计算	PS 端直接读寄存器，不依赖 DMA 收满
静音识别	无拍约 2s 后归零	静音门 + 看门狗协同
音乐节奏检测	表现良好	连续底鼓/镲片场景稳定识别节拍
节拍器单音信号	实时链持续优化中	连续实时检测精度仍在改进（如实说明）

注：受输入音量、采样连续性等因素影响，节拍器单一音色信号的连续实时读数稳定性仍有优化空间；对音乐节奏的识别作为本作品主要验证方向表现良好。



1.5 主要创新点

① 架构创新：采用“PL 搬运 + PS 计算”软硬分离思路，既保证实时性又保留算法可迭代性，避免重复综合。

② 算法创新：能量检测 + 区间众数 + 指数遗忘，在抑制 off-beat 误检与快速节奏切换之间取得平衡。

③ 工程创新：通过“DMA 块长与 FIFO 深度匹配（LEN=512）+ 超时取数”的做法，系统性解决 AXI-DMA 收帧死锁这一常见疑难。

1.6 设计流程

开发采用“HLS 设计 → C 仿真 → 高层次综合 → IP 打包 → Vivado 块设计集成 → 生成 bitstream → PYNQ 上板实测 → 实时可视化”的闭环流程。任何算法改动先经 C-sim

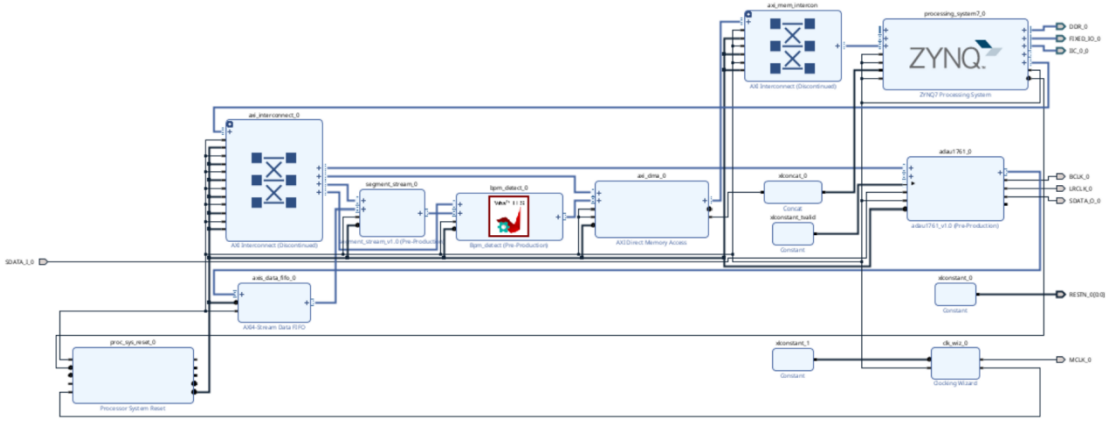
与 Python 镜像标定，然后再上板实测；为便于调试，在 bpm_detect IP 中暴露 BPM、节拍计数与包络能量三个只读寄存器，供 PYNQ 脚本在线观测。

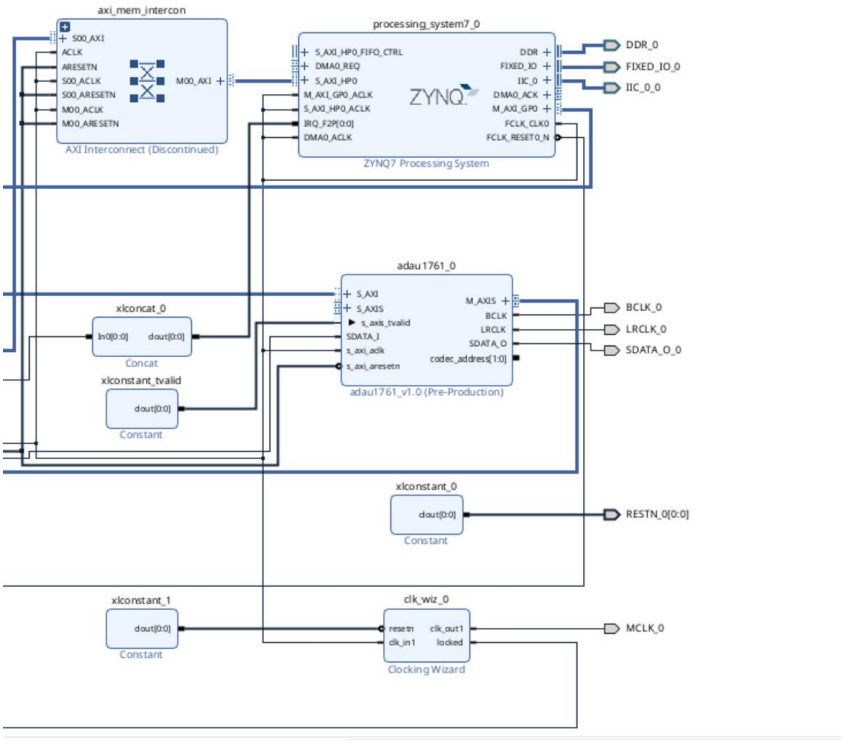
第二部分　系统组成及功能说明

2.1 整体介绍

系统整体由信号前端、可编程逻辑（PL）数据处理与处理器系统（PS）软件三部分构成，数据流为：ADAU1761 编解码器（模拟输入→数字）→adau1761 接口 IP→axis_data_fifo → 自定义字数闸门（segment）→bpm_detect 检测 IP→AXI DMA→DDR 内存 →PS 端 PYNQ/Python 读取与可视化。

其中 PL 完成“采样、搬运、检测”三件事，PS 完成“算法结果读取与波形呈现”。BPM 由 PL 端自由运行（freerun）的 IP 持续计算并写入寄存器，PS 直接读取即可，不依赖 DMA 是否收满，二者完全解耦。





2.2 硬件系统介绍

2.2.1 硬件整体介绍。

硬件平台为 Xilinx PYNQ-Z2 开发板，主芯片为 Zynq-7020（xc7z020clg400-1）。音频前端使用板载 ADAU1761 立体声编解码器，承担话筒/线路输入的模数转换与 I2S 时隙输出。FPGA 侧的 adau1761 IP 从接口分频产生 I2S 主时序，将数字音频交给 AXI-Stream 数据链。

时钟链：PS FCLK0=100MHz → 时钟生成器产生 25MHz MCLK 送编解码器，同时 FPGA 分频产生约 48.8kHz 帧率与时隙时钟，编解码器作为 I2S 从机。整个 PL 采用单一 100MHz 时钟域。

2.2.3 电路各模块介绍

核心电路模块为 ADAU1761 编解码器与相关模拟输入通路。板载 MIC 座（MIC1/MIC2）接耳机线麦克风，MICBIAS 经上拉电阻供电；也支持线路输入（LINE-IN）。编解码器通过 I2C（地址 0x3B）配置，通过 I2S 与 FPGA 交换音频数据。

关键 I2C 寄存器配置包括：R0=0x03（使能核心时钟、512×fs 直通）、R65=0x7F 与 R66=0x01（时钟与各模块使能）、输入音量/混频与 ADC 控制寄存器。编解码器无自动初始化，必须在上电后主动写入时钟使能再配置增益，否则左声道恒满幅、无有效信号。

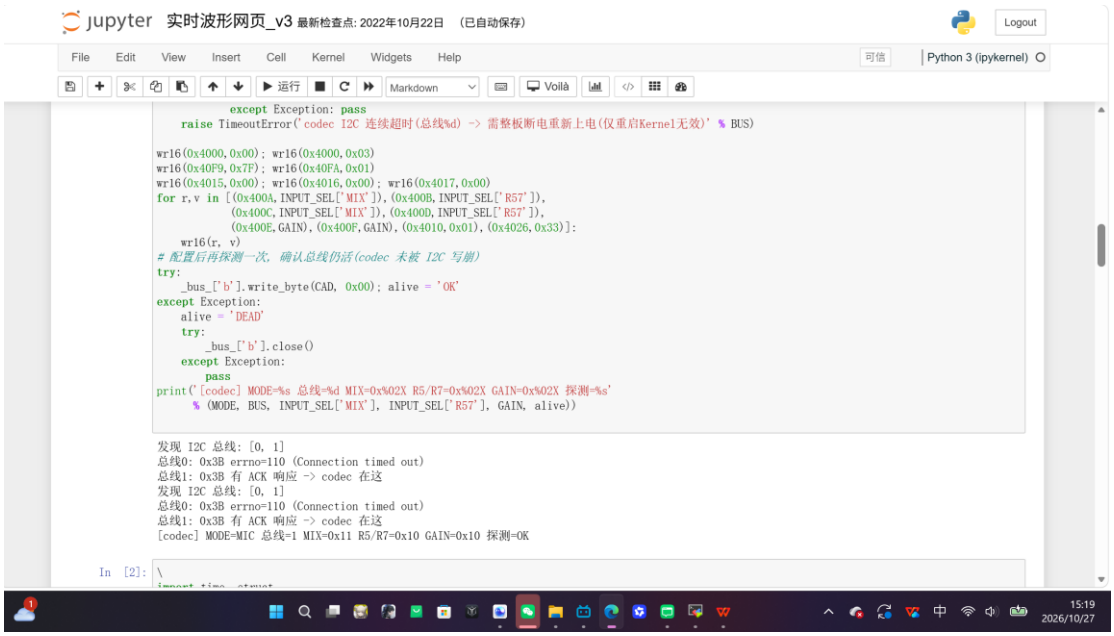
2.3 软件系统介绍

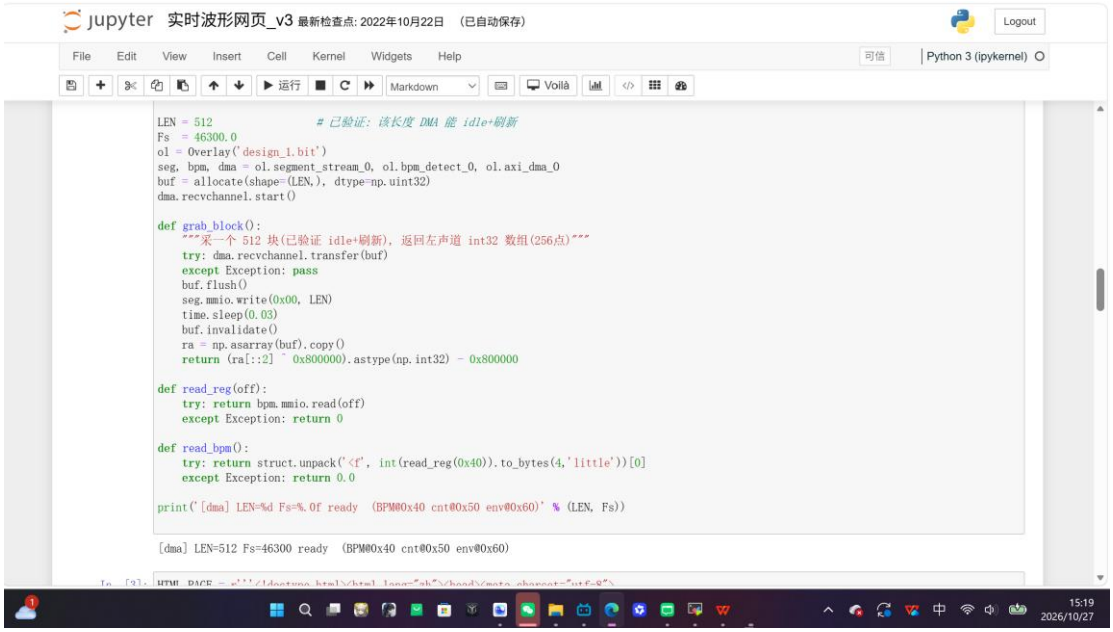
软件侧运行于 PYNQ 的 Jupyter 环境中，使用 Python 与 PYNQ Overlay API。软件完成三件事：编解码器 I2C 配置、DMA 音频采集、波形与 BPM 的实时可视化。整体采用“每块采集、每块刷新”的流水方式，配合缓存一致性操作（flush/invalidate）保证读到最新数据。

① I2C 编解码器配置模块：使用 smbus2 库，编解码器 7 位地址 0x3B、16 位寄存器地址。写用 write_i2c_block_data；读必须用 i2c_rdwr 保持 restart（否则丢 restart 恒返回假值）。初始化流程为“先时钟使能 → 再增益 → 再模拟验证”。

② DMA 采集模块：分配内存缓冲，采用“transfer → 缓存失效 → 写闸门放行 → 等待 → 读取”的流程逐块采集并拼接成完整波形。

③ 可视化模块：将 24bit 采样解包为带符号波形在 G1 绘制；G2 持续读取 BPM 寄存器绘制实时曲线，并在判定静音时显示 0。

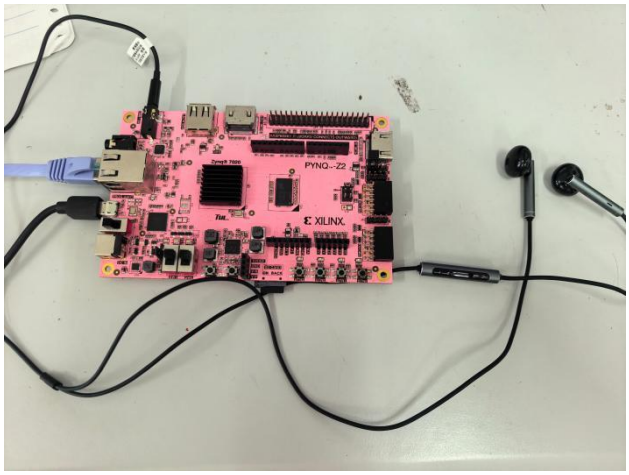




第三部分 完成情况

已完成端到端系统联调：从麦克风/音频输入、编解码器数字化、PL 数据通路、DMA 采集，到 PS 端 BPM 计算与实时双波形可视化，整条链路实测打通。系统在真实音乐节奏检测上表现良好，静音自动归零。

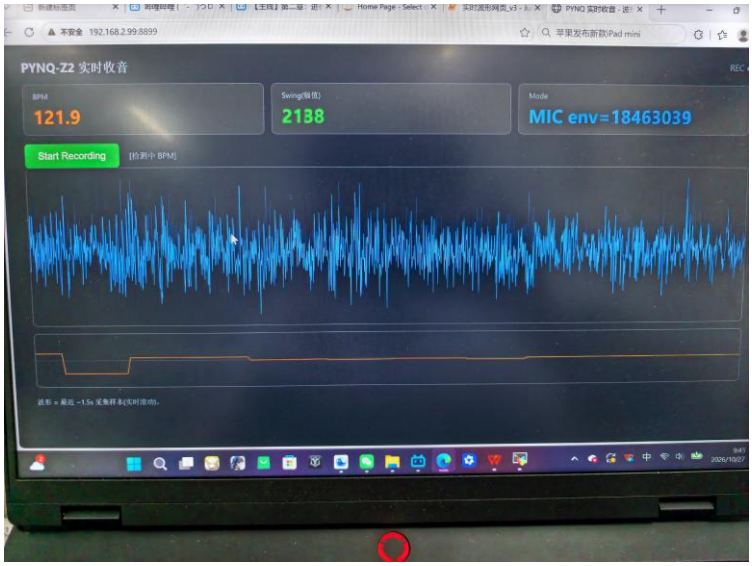
3.1 工程成果（分硬件实物、软件界面等设计结果）



基于 PYNQ-Z2 平台完成编解码器配置与完整数据链路搭建：ADAU1761 经 I2C 正确初始化，I2S 数据通路、AXI-Stream FIFO、自定义闸门、DMA 采集全部打通。通过 I2C 读回校验、模拟路径幅度验证与 DMA 硬通路验证三项判据确认链路健康。

已实现 Jupyter 网页端实时双波形：G1 显示原始音频波形，G2 显示 BPM 实时曲线；

可显示当前 BPM、幅值（Swing）与检测模式，并支持停止录音控制。



硬件链路稳定性：编解码器、I2C、I2S、DMA、采样保存全链路已实测打通并稳定，能够连续采集真实节拍器信号（如 120 BPM）并保存波形。

算法验证成果：C 仿真在干净信号下 120 BPM 可读 ≈120（emode 版实测 120.6）；Python 镜像逐行复刻硬件行为，用于离线标定参数并复现故障。

当前进度说明：在强脉冲节拍器、高动态范围音频场景下，阈值自适应仍需进一步迭代（存在大峰垫高慢均值阈值导致漏拍的现象）；在常规音乐场景下检测表现良好。该问题已定位至阈值机制，属于后续优化重点。

关键调试经验沉淀：DMA 传输长度需与上游 FIFO 深度匹配、I2C 读取需保持 restart、网页波形须与硬件计算位数口径一致（避免观测面与执行面不一致）。

第四部分 总结

4.1 可扩展之处

未来可在以下方向扩展：

① 算法层面：将自适应阈值升级为峰值跟踪/归一化阈值，进一步提升高动态信号与强脉冲节拍器的检测鲁棒性；可尝试能量增量判别以免疫音量变化。

② 参数可重构：将 ADAPT、静音门限、不应期等参数接到运行时寄存器，实现上位机在线调参，免去为每次试值重新综合。

③ 功能扩展：接入心率/搏动检测、音乐 BPM 自动标注、多通道输入切换等。

④ 工程化：将 Jupyter 脚本打包为独立应用，提升部署与演示便捷性。

4.2 心得体会

从零搭建这套基于 PYNQ 的实时音频 BPM 检测系统，回头看，最大的体会是：FPGA 工程里“观测面与执行面不一致”会带来大量无效调试。早期浏览器上看到的波形虽然看着干净，但硬件检测器实际消费的却是经过 16 位截断的折叠噪声，两者根本不是一个信号，结果就是多次“盲调”都没有结果。后来把 HLS 算法逐行镜像成 Python，再和综合后的 RTL 逐项对账，才确认综合本身是干净的，问题出在源码参数与前端处理上，从此转向数据驱动标定。

另一个深刻教训是数据链路的“长度适配”。AXI DMA 采集长期看起来有反应却不通，根因竟然是传输长度远大于上游 FIFO 深度，导致 DMA 永远等不满、通道永不空闲。把块长定为与 FIFO 深度一致的 512 后，一下就打通了。这也让我们学会用“小帧 vs 大帧收到字数对比”快速定位硬件边界行为，而不是反复烧板碰运气。

算法上，我们体会到静态阈值面对真实音频往往无能为力：音量一大，阈值跟着水涨船高，真拍被削、读数失真。自适应阈值、静音门与看门狗归零的组合，以及用区间众数等权投票替代能量加权，都是在一次次板上实测数据的驱动下逐步收敛出来的。真正把“能用”和“好用”区分开的，往往就是这些不起眼却决定成败的工程细节。

最后，得益于整个调试过程中形成的这套习惯：先离线镜像验证、再上板实测，用双保险确认病根。

第五部分 参考文献

[1] Pan J, Tompkins W J. A real-time QRS detection algorithm. IEEE Trans. Biomed. Eng., 1985, 32(3): 230-236.

[2] Xilinx. PYNQ: Python Productivity for Zynq[EB/OL]. https://www.pynq.io/.

[3] Xilinx. Vitis HLS User Guide (UG1399). 2024.

[4] Analog Devices. ADAU1761 Datasheet: SigmaDSP Stereo, Low Power, 96 kHz, 24-Bit Audio Codec.

[5] Xilinx. AXI DMA v7.1 LogiCORE IP Product Guide (PG021).

[6] Xilinx. Vivado Design Suite User Guide: Designing with IP (UG896).

[7] Scheirer E D. Tempo and beat analysis of acoustic musical signals. J. Acoust. Soc. Am., 1998, 103(1): 588-601.